

Lab 13 — Connectivity: Wi-Fi Station and JSON Telemetry

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	3 hrs
Introduces	event loops, the network stack, HTTP
Prerequisites	Lab 7 (something worth sending).
Beyond the board	A 2.4 GHz Wi-Fi network and a host PC to run the receiver.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§13.1 objective**, **§13.2 parts and wiring**, **§13.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

13.1 Objective & TM4C bridge

Get the board onto your network and have it send a real reading to a server you run. This is the step that turns a sensor into a connected device.

TM4C BRIDGE

None, honestly — the TM4C had no radio and no network stack. Everything here is new territory, and the mental shift worth naming is from **sequential** to **event-driven**: you no longer call a function and get an answer, you register interest and get called back when the network is ready.

13.2 Parts & wiring

Item	Status	Notes
ESP32-S3 board	radio is on the module	No antenna to attach; it is on the PCB.
A 2.4 GHz Wi-Fi network	Required	2.4 GHz only. The ESP32-S3 cannot see a 5 GHz network, and this catches people whose router hides both bands behind one name.

Item	Status	Notes
A host PC running a small HTTP server	Required	One command: <code>python -m http.server 8000</code> for a quick test, or a small Flask app to actually receive POSTs.

No wiring. But you do need two pieces of information before starting: your network's SSID and password, and **your laptop's IP address on that network** — both machines must be on the same network for the board to reach it. Find the laptop's IP with `ipconfig` on Windows.

WARNING

Do not commit your Wi-Fi password

This lab uses `menuconfig` to hold the credentials, which keeps them in `sdkconfig`. That file goes into version control. For a throwaway lab it is fine; for anything you publish to GitHub it is not. The professional pattern is to read credentials from NVS at runtime and provision them separately — which is one reason Lab 10 came first.

13.3 How it works

This is the first lab where your board talks to something that is not on your desk. Assume no networking background; it is all below.

What "getting on Wi-Fi" actually involves

Connecting is not one step, it is four, and knowing them turns a mysterious failure into a diagnosable one.

1. **Scan.** The radio listens for beacon frames that access points broadcast, building a list of nearby networks and their signal strengths.
2. **Authenticate.** Prove you know the password. With WPA2 this is a four-way handshake that derives a session key without ever sending the password itself.
3. **Associate.** The access point accepts you as a client and assigns you a slot. You now have a *link* — but no address.
4. **DHCP.** You broadcast a request and a server hands back an IP address, a gateway and a DNS server. Only now can you open a socket.

That fourth step is the one people forget. **Being connected to Wi-Fi and having an IP address are different states**, and code that starts sending the moment it sees "connected" will fail. The event you actually wait for is `IP_EVENT_STA_GOT_IP`.

STA stands for **station** — a client that joins someone else's network. The alternative is **AP** (access point) mode, where the ESP32 creates its own network for others to join, which is how devices offer a "connect to my Wi-Fi to configure me" setup flow. This lab uses STA.

Event	Meaning
<code>WIFI_EVENT_STA_START</code>	driver is up, begin connecting
<code>WIFI_EVENT_STA_CONNECTED</code>	associated with the access point, but you have NO IP yet
<code>IP_EVENT_STA_GOT_IP</code>	DHCP finished. Only NOW can you open a socket.
<code>WIFI_EVENT_STA_DISCONNECTED</code>	fires whenever the link drops. YOU must reconnect.

A device that never handles the last one stays offline forever after a router reboot. That is the difference between a demo and a product.

The dependency that surprises people is the first one: **Wi-Fi requires NVS**. The radio needs per-chip calibration data — RF characteristics vary between individual parts — and the driver stores it in the NVS partition rather than recomputing it on every boot. Skip `nvs_flash_init()` and Wi-Fi refuses to start with an error that does not obviously say "your storage is not mounted." That is precisely why Lab 10 comes before this one.

Synchronous versus event-driven, and why networking forces the change

Every peripheral so far was **synchronous**: you called a function, and it either returned an answer or timed out, right there in your code. `i2c_master_transmit_receive` behaves like that, and it is a comfortable way to program.

Networking cannot work that way, for a specific reason: **the interesting events do not happen when you ask for them**. Association might take 200 ms or 8 seconds. DHCP might fail and retry. And crucially, the link can drop *three hours later* when someone reboots the router, at a moment when your code is not asking about anything.

So ESP-IDF uses an **event loop**: a dispatcher task that receives events from the drivers and calls whichever handlers registered interest. Your code says "when an IP arrives, run this" rather than "wait here until an IP arrives." It is the same inversion as an interrupt handler in Lab 8 — you register a callback and the system calls you — applied to software events instead of hardware ones.

NOTE

What `example_connect()` is, and what it is hiding

`example_connect()` is **not part of the ESP-IDF API**. It is a helper that lives in the examples folder of your own installation, at `examples/common_components/protocol_examples_common`. It registers the handlers, starts Wi-Fi, and blocks until an IP is assigned, so that learning code can pretend the whole thing is synchronous.

It is the right choice while you are learning one thing at a time. It is the wrong choice for anything real, because it has no reconnect policy — and a device that stops working permanently when a router reboots is broken. **Open that source file and read it**. It is short, and it is the clearest example you have of writing the handlers yourself.

What you actually send, and why JSON

Your board has two floats. You could send them as eight raw bytes, which is compact and fast — but then both ends must agree exactly on byte order, float format and field order, and nothing else can read the data without your documentation.

JSON is text that describes itself: `{"rms":0.0123,"peak":0.0456}`. Every server language parses it without effort, you can read it with your own eyes while debugging, and adding a third field later does not break an existing receiver. For telemetry at a few messages per second, that costs you maybe 30 bytes and buys enormous convenience.

The trade reverses at high rates. Streaming 1 kHz samples as JSON would waste most of your bandwidth on punctuation, and there you would use a binary format. Knowing *why* you chose one is the point, not the choice itself.

HTTP in one paragraph

HTTP is a request-response protocol carried over TCP. A request has a **method**, a **path**, **headers**, and optionally a **body**. The method declares intent: **GET** means "give me something," **POST** means "here is data for you." Sending a sensor reading is a POST. The server replies with a **status code** — 200 for success, 404 if the path does not exist, 500 if the server itself failed — and reading that code tells you which side went wrong.

The `Content-Type` header matters more than it looks: it tells the server how to interpret the body. Send JSON without declaring `application/json` and a strict server will reject a perfectly good request.

Security you should be aware of even in a lab

Two things this lab does that you would not ship.

Plain HTTP. The data crosses your network unencrypted and unauthenticated. Anyone on the same Wi-Fi can read it or forge it. Real telemetry uses HTTPS, which is exactly what Lab 15 sets up for firmware downloads — and firmware is the one place where getting this wrong is catastrophic rather than embarrassing.

Credentials in `sdconfig`. Putting your Wi-Fi password in `menuconfig` writes it into a file that goes into version control. Fine for a throwaway lab, not fine for anything you publish. The professional pattern is to store credentials in NVS and provision them separately — over BLE, over a temporary AP-mode setup page, or at the factory. §13.7 has you do exactly that.

13.4 Procedure & code

1. `idf.py menuconfig` → **Example Connection Configuration** → enter your SSID and password.
2. Start a receiver on your laptop and note its IP.
3. Type the code, set the URL to your laptop's IP, flash.

```
#include "esp_netif.h"
#include "esp_event.h"
#include "nvs_flash.h"
#include "protocol_examples_common.h" // provides example_connect()

void app_main(void) {
    ESP_ERROR_CHECK(nvs_flash_init());           // Wi-Fi keeps calibration in NVS
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    ESP_ERROR_CHECK(example_connect());           // blocks until STA has an IP
```

```
// ... now you're online; call post_reading() with real feature values ...
}
```

The part worth typing in full is the POST itself, because this is the code you will actually own:

```
#include <stdio.h>
#include "esp_http_client.h"
#include "esp_log.h"

static void post_reading(float rms, float peak) {
    char body[96];
    int n = snprintf(body, sizeof(body), "{\"rms\":%.4f,\"peak\":%.4f}", rms, peak);

    esp_http_client_config_t cfg = {
        .url = "http://192.168.1.50:8000/telemetry", // your laptop's IP + port
        .method = HTTP_METHOD_POST,
    };
    esp_http_client_handle_t c = esp_http_client_init(&cfg);
    esp_http_client_set_header(c, "Content-Type", "application/json");
    esp_http_client_set_post_field(c, body, n);

    if (esp_http_client_perform(c) == ESP_OK)
        ESP_LOGI("lab13", "POST -> HTTP %d", esp_http_client_get_status_code(c));
    else
        ESP_LOGE("lab13", "POST failed");
    esp_http_client_cleanup(c);
}
```

13.5 Code walkthrough, line by line

Two blocks. The first is four lines of setup that hide a great deal; the second is the part you will actually own and modify.

Block 1, line 4 · `#include "protocol_examples_common.h"`

The one include here that is **not** a real ESP-IDF API. It comes from the examples tree in your own installation, and it is what provides `example_connect()`. If you copy this code into a project that does not pull in that component, it will not build — which is a good early reminder that not everything in an example is production API.

Block 1, line 7 · `ESP_ERROR_CHECK(nvs_flash_init());`

Mounts the NVS partition, and it must come first because the Wi-Fi driver stores per-chip radio calibration there. Note this version is not wrapped in the recovery `if` from Lab 10 — on a device that has already run, NVS is mounted cleanly and this succeeds. In real firmware you would keep the full recovery block.

Block 1, line 8 · `ESP_ERROR_CHECK(esp_netif_init());`

Starts the TCP/IP stack — ESP-IDF uses lwIP, a small open-source implementation designed for embedded systems. This is the software that turns "send these bytes to that IP address" into actual packets with headers, sequence numbers and retransmission. It is a substantial piece of software, several tens of kilobytes, and it is why networking costs you flash and RAM.

Block 1, line 9 · `ESP_ERROR_CHECK(esp_event_loop_create_default());`

Creates the system event loop described in §13.3: a dedicated task that receives events from drivers and dispatches them to registered handlers. "Default" means the shared system-wide loop, as opposed to a private one you could create for your own events.

Nothing visible happens on this line. It is infrastructure — but without it, the Wi-Fi driver has nowhere to report "connected" or "got IP" and the next line would hang forever.

Block 1, line 10 · `ESP_ERROR_CHECK(example_connect());`

Starts Wi-Fi and **blocks until the chip has an IP address**. Everything from §13.3 — scan, authenticate, associate, DHCP — happens inside this call, driven by events, while your task sits waiting on a semaphore the helper set up.

The reason it can block when the underlying process is event-driven is worth noticing: the helper registers handlers, then waits on a synchronisation primitive that its own `GOT_IP` handler gives. That is the standard way to put a synchronous face on an asynchronous system, and it is the same pattern as Lab 7's semaphore handoff.

It also reads the SSID and password you set in `menuconfig`, which is where the security caveat in §13.3 comes from.

Block 2, line 5 · `char body[96];`

A fixed buffer for the JSON text. **Fixed, on the stack, sized generously** — the pattern you want in firmware, because it cannot fail at runtime the way a heap allocation can, and it cannot fragment memory. Ninety-six bytes is far more than the roughly thirty this message needs, and that headroom is deliberate.

Block 2, line 6 · `int n = snprintf(body, sizeof(body), "{\"rms\":%.4f,\"peak\":%.4f}", rms, peak);`

Three separate things worth slowing down for.

The escaped quotes. JSON requires double quotes around its keys, and you are writing that JSON *inside* a C string literal which is itself delimited by double quotes. So a literal `"` in the output must be written `\"` in the source, telling the compiler "this quote is data, not the end of my string." The format string `"{\"rms\":%.4f}"` produces the text `{"rms":0.0123}`. This is the single most common source of confusion when generating JSON from C, and it disappears the moment you see what the backslashes are doing.

snprintf, not sprintf. `sprintf` writes as many characters as the output needs and has *no idea* how big your buffer is; if the numbers run long it writes straight past the end, corrupting whatever sits next to it on the stack — often the return address, which is how buffer overflows become security vulnerabilities. `snprintf` takes the buffer size and truncates instead. **Prefer the bounded version of every string function, always.** This is the cheapest security habit in C and it is a standard interview topic.

The return value. `n` is the number of characters written, which line 13 needs as the body length. Note a subtlety worth knowing: if the output *was* truncated, `snprintf` returns the length it *would have* written, not the length it actually did — so on a truncation, `n` can exceed `sizeof(body)`. Robust code checks for that.

`%.4f` gives four decimal places, enough to see a still board's small noise floor.

Block 2, lines 8-11 · `esp_http_client_config_t cfg = { .url = ..., .method = HTTP_METHOD_POST };`

`.url` is the full destination including scheme, host, port and path:

`http://192.168.1.50:8000/telemetry`. Every part matters — wrong port and nothing listens, wrong path and you get a 404.

You must use your laptop's **actual IP on that network**, not `localhost` and not `127.0.0.1`. Those mean "this machine" and from the ESP32's point of view, "this machine" is the ESP32. Find the real address with `ipconfig`.

`.method = HTTP_METHOD_POST` declares intent: you are sending data, not requesting it.

Block 2, line 12 · `esp_http_client_handle_t c = esp_http_client_init(&cfg);`

Creates a client instance from the config. The familiar handle pattern — and note it **allocates memory**, which is why line 19 must free it.

Block 2, line 13 · `esp_http_client_set_header(c, "Content-Type", "application/json");`

Adds an HTTP header declaring the body format. Omit it and many servers will reject or misparse a perfectly valid request, because the default assumption is form-encoded data rather than JSON.

Block 2, line 14 · `esp_http_client_set_post_field(c, body, n);`

Attaches the payload and its length. The length is passed explicitly rather than being measured with `strlen`, which means the body does not have to be text at all — the same call works for binary payloads.

Block 2, line 16 · `if (esp_http_client_perform(c) == ESP_OK)`

The line that actually does everything: opens a TCP connection, sends the request and headers and body, waits for the response, and reads it. It is **synchronous and can take a long time** — hundreds of milliseconds on a busy network, or the full timeout if the server is unreachable.

That is why you would never call this from a timer callback or an ISR, and why in a real design the network work belongs in its own task at modest priority, so a slow POST cannot delay your sampling. Lab 7's task structure is what makes that easy.

`ESP_OK` means the HTTP exchange completed — note carefully that it does **not** mean the server liked your request. A 404 is a successful exchange with an unsuccessful outcome, which is why the next line checks the status separately.

Block 2, line 17 · `esp_http_client_get_status_code(c)`

Returns the HTTP status. Reading it is what separates "the network worked" from "the request worked," and the distinction is diagnostic: a 200 means everything is fine, a 404 means your networking is perfect and your URL path is wrong, a 500 means the server itself failed, and no response at all points at the IP, the port or a firewall.

Block 2, line 19 · `ESP_LOGE("lab13", "POST failed");`

Note the **E** for error rather than the **I** used on success. Log levels are not decoration — you can filter by them at runtime, and on a device that has been running for hours the ability to grep for errors is the difference between finding a fault and scrolling forever.

Block 2, line 20 · `esp_http_client_cleanup(c);`

Frees the client and everything it allocated. **Forgetting this leaks memory on every single call**, which is a particularly nasty class of bug: the device works perfectly on the bench for an hour, then fails overnight when the heap is exhausted. `esp_get_minimum_free_heap_size()` is how you catch it: log the low-water mark periodically and watch for a downward trend that never recovers.

Note it sits *outside* the `if`, so it runs on both the success and the failure path. Cleanup that only happens when things go well is not cleanup.

13.6 Docs & references

DOCS

Trace every line to its source

`esp_http_client_*` — API Reference → Protocols → **ESP HTTP Client**. Read about the event handler callback, which is how you would stream a large response rather than buffering it.

`esp_netif_init`, `esp_event_loop_create_default` — API Reference → Network and System. The **Event Handling** page is the one to read properly if you want to understand the model rather than just call the helper.

`example_connect` — this is *not* a real ESP-IDF API. It lives in `examples/common_components/protocol_examples_common` in your own installation. Open that source and read it; it is a short, clear example of writing the event handlers yourself, and it is the natural next step.

Wi-Fi driver — API Reference → Network → **Wi-Fi**, for when you replace the helper with your own station setup and reconnect logic.

`snprintf` — any C reference. Note carefully what it returns when the output is truncated; it is not what people assume.

13.7 Verify, troubleshoot, your turn

Verify. The log shows the connection sequence ending in an assigned IP address, then `POST -> HTTP 200`. On your laptop, the server prints the received JSON. Seeing your own sensor's numbers appear in a terminal on another machine is the moment the project stops being a board on a desk.

Troubleshoot

Symptom	Cause and fix
Never connects	5 GHz network, or a typo in the credentials. Confirm the band first.
Connects but the POST fails	Wrong laptop IP, wrong port, or the laptop's firewall is blocking inbound connections. The firewall is the usual culprit.

Symptom	Cause and fix
HTTP 404	The board reached the server but the path does not exist. Your networking is fine; fix the URL or the route.
Wi-Fi init fails outright	NVS was not initialised first.
Works, then stops after hours	Missing <code>cleanup</code> leaking memory, or no reconnect handler after a router blip.

Your turn

1. **Send real data.** Wire this into Lab 7 so the feature task posts its actual rms and peak. That is the full **acquire → process → communicate** pipeline running end to end, and it is genuinely demonstrable at a career fair.
2. **Handle disconnection.** Replace `example_connect()` with your own event handlers and add reconnect-with-backoff. Then reboot your router while the board runs and confirm it recovers on its own. A device that survives that is meaningfully more real than one that does not.
3. **Move the credentials into NVS** so they are provisioned rather than compiled in, and remove them from `sdkconfig`. This closes the security gap flagged in §13.2 and combines two labs into something that looks like a product.

Errata and corrections

Defects confirmed against the hardware and the vendor documentation, with the evidence that established each one. They are printed rather than quietly patched because how an error was caught is usually more instructive than the error.

the verification server cannot accept your POST

WARNING

Confirmed defect. Severity: the lab cannot succeed as written.

§13.2 tells you to run `python -m http.server 8000` and §13.7 says the server will print your JSON. **It cannot.** Python's `SimpleHTTPRequestHandler` implements only `do_GET` and `do_HEAD`. A POST receives **HTTP 501 Unsupported method**.

Evidence: tested directly. A POST to `python -m http.server` returns `HTTP 501`; a GET to the same server returns `HTTP 200`. Inspecting the class confirms `hasattr(SimpleHTTPRequestHandler, 'do_POST')` is `False`.

What you would have seen: the board logs `POST -> HTTP 501`, not the 200 the lab promises, and nothing is printed on the laptop. Note the board's own code is *correct* — the failure is entirely in the host instructions.

Fix: use the receiver supplied in **Part B** below, which implements `do_POST`, prints the body and returns JSON. Part B also supplies the retry, backoff and reconnect logic that §13.7 leaves as an exercise.

Bench realities

Problems that stop this lab but are not about the peripheral it teaches. Each one cost real time on a real bench.

The Kconfig menu the lab tells you to open does not exist

WARNING

Symptom: Lab 13 says go to "Example Connection Configuration" in menuconfig. There is no such menu.

Cause: `example_connect()` lives in `protocol_examples_common`, which is a component in the ESP-IDF `examples` tree, not part of the SDK your project sees. A blank project has no idea it exists, so neither its code nor its Kconfig menu is present.

Fix — add this to your **top-level** `CMakeLists.txt`, before the `include()` line:

```
cmake_minimum_required(VERSION 3.16)

set(EXTRA_COMPONENT_DIRS
    $ENV{IDF_PATH}/examples/common_components/protocol_examples_common)

include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(lab13_wifi)
```

Then `idf.py fullclean` and re-run `menuconfig`. The menu appears, and the SSID and password fields with it.

The ordering matters: `project.cmake` reads `EXTRA_COMPONENT_DIRS` when it runs, so setting it afterwards has no effect. This is the single most common reason Lab 13 stalls before it even starts.

Your Wi-Fi is 5 GHz only

WARNING

Symptom: the ESP32-S3 never associates, and the network is not even in a scan.

Cause: the ESP32-S3 radio is **2.4 GHz only**. Many modern access points, and most apartment networks you do not control, run 5 GHz — sometimes with WPA3-only security, which is a separate setting to check — the ESP32-S3 *does* support WPA3, so that alone is not the blocker, but a mixed or misconfigured mode can still refuse it.

Fix that needs no router access: use a phone hotspot. On iPhone, **Settings → Personal Hotspot → Maximize Compatibility ON**. That toggle is what forces 2.4 GHz and WPA2-Personal, and without it the hotspot is 5 GHz and equally invisible.

Then put your laptop on the same hotspot and find its address with `ipconfig` — it will be something like `172.20.10.3`, with the board landing nearby. That laptop address is what goes in the URL in Part B.

Windows Firewall silently drops the board's POST

WARNING

Symptom: the board reports a timeout or connection failure, the receiver prints nothing, and everything about the code looks right.

Cause: your receiver is listening, but Windows Firewall blocks the *inbound* connection from the board. Nothing on either side reports this — the board just times out.

Fix, PowerShell as Administrator:

```
New-NetFirewallRule -DisplayName "ESP32 telemetry 8000" `
  -Direction Inbound -Protocol TCP -LocalPort 8000 -Action Allow
```

A quick way to tell this apart from a code problem: from the laptop itself, `curl -X POST http://127.0.0.1:8000/telemetry -d '{}'`. If that works and the board does not, the code is fine and the network path is not.

Calling `example_connect()` twice aborts the boot

WARNING

Symptom: `assert failed ... duplicate key in esp_netif_new_api, immediately at startup.`

Cause: `example_connect()` registers a network interface with a unique key. Call it a second time — easy to do when you paste the setup block into a task as well as into `app_main` — and the second registration collides.

Fix: call it exactly once, from `app_main`, before anything that needs the network. The error message names the real cause, which is unusually helpful; the difficulty is only that "duplicate key" does not obviously mean "you connected twice."

Do not commit your Wi-Fi password

WARNING

It is sitting in plaintext in `sdkconfig` right now

Setting the SSID and password through `menuconfig` writes them into `sdkconfig`, which is a file people routinely commit. If you push a lab project to GitHub as portfolio evidence — which is the whole point of building them — your network credentials go with it.

Add to `.gitignore`:

```
sdkconfig
sdkconfig.old
build/
```

Commit `sdkconfig.defaults` instead, with the non-secret settings only. And if you have already pushed a password, rotate it — deleting the file does not remove it from the repository history.

An interviewer who looks at your GitHub will notice either way. Getting this right is a small, visible signal of security awareness; getting it wrong is a more visible one.

Part B — Wi-Fi Telemetry That Actually Verifies

Replaces: Lab 13 §13.2 and §13.7 · **Fixes:** a host server that cannot accept POST · **Adds:** retry, backoff, reconnect

13B.1 What was wrong

Lab 13 asks you to run `python -m http.server 8000` and says the server will print your JSON. Python's `SimpleHTTPRequestHandler` serves files and implements only `do_GET` and `do_HEAD`. Your POST receives **HTTP 501 Unsupported method** and nothing is printed.

The board's firmware was correct. Only the host instruction was wrong — which is worth noticing, because it is exactly the class of bug that makes you doubt working code.

13B.2 The receiver

Save this as `telemetry_receiver.py` next to your project and run `python telemetry_receiver.py 8000`. It has no dependencies beyond the standard library.

```
import json, sys
from http.server import BaseHTTPRequestHandler, HTTPServer
class H(BaseHTTPRequestHandler):
    def do_POST(self):
        n = int(self.headers.get('Content-Length', 0))
        raw = self.rfile.read(n)
        try:
            obj = json.loads(raw); ok = True
        except Exception:
            obj = raw.decode('utf-8', 'replace'); ok = False
        print(f"{self.client_address[0]} {self.path} {obj}", flush=True)
        body = json.dumps({"ok": ok}).encode()
        self.send_response(200 if ok else 400)
        self.send_header('Content-Type', 'application/json')
        self.send_header('Content-Length', str(len(body)))
        self.end_headers(); self.wfile.write(body)
    def log_message(self, *a): pass
if __name__ == '__main__':
    port = int(sys.argv[1]) if len(sys.argv) > 1 else 8000
    print(f"telemetry receiver on :{port} (ctrl-c to stop)", flush=True)
    HTTPServer(('0.0.0.0', port), H).serve_forever()
```

KEY POINT**Verified output**

Tested against a valid POST and a malformed one:

```
text $ python telemetry_receiver.py 8898 telemetry receiver on :8898 (ctrl-c to stop)
127.0.0.1 /telemetry {'rms': 0.0123, 'peak': 0.0456} <- HTTP 200 127.0.0.1 /telemetry not json
<- HTTP 400
```

Returning **400 on a body it cannot parse** matters: it means a malformed payload from the board is visible as a status code rather than silently accepted, which is what lets §13B.4's failure counter mean anything.

13B.3 Line-by-line, the parts that are not obvious

do_POST · the method name is the dispatch mechanism

`BaseHTTPRequestHandler` dispatches by looking for a method named `do_` plus the HTTP verb. Defining `do_POST` is the entire reason this server accepts POST and the stock one does not. Add `do_PUT` and you would handle PUT.

Content-Length · `n = int(self.headers.get('Content-Length', 0))`

HTTP does not delimit a body; you must be told its length and read exactly that many bytes. Reading fewer leaves data in the socket and desynchronises the next request; reading more blocks forever. This is the same framing problem as the UART start/stop bits in Lab 3, one layer up.

the response · `send_response / send_header / end_headers / write`

The order is fixed: status line, then headers, then a blank line, then the body. Sending `Content-Length` lets the ESP32's HTTP client know when the response is complete instead of waiting for the connection to close — which is why the board's `esp_http_client_perform()` returns promptly rather than sitting until its timeout.

flush=True · `print(..., flush=True)`

Python buffers stdout when it is not a terminal. Without this, piping the output to a file shows nothing until the buffer fills, which looks exactly like a board that is not transmitting.

13B.4 The firmware side Lab 13 left as an exercise

§13.7 asked you to add reconnect and backoff. Here it is, because a device that gives up permanently on the first failure is not a useful demonstration.

```
#include <inttypes.h>

static uint32_t g_post_ok, g_post_fail;           // observable counters, not just logs

static esp_err_t post_once(const char *url, const char *body, int len) {
    esp_http_client_config_t cfg = {
        .url = url,
        .method = HTTP_METHOD_POST,
        .timeout_ms = 5000,                       // BOUNDED. never let a POST hang forever
    };
    esp_http_client_perform(&cfg);
}
```

```

};
esp_http_client_handle_t c = esp_http_client_init(&cfg);
if (!c) return ESP_FAIL;
esp_http_client_set_header(c, "Content-Type", "application/json");
esp_http_client_set_post_field(c, body, len);

esp_err_t err = esp_http_client_perform(c);
if (err == ESP_OK) {
    int status = esp_http_client_get_status_code(c);
    if (status < 200 || status >= 300) err = ESP_FAIL;    // 501 and 404 are NOT success
}
esp_http_client_cleanup(c);    // on BOTH paths, or you leak per call
return err;
}

static void post_with_backoff(const char *url, const char *body, int len) {
    int delay_ms = 500;
    for (int attempt = 0; attempt < 5; attempt++) {
        if (post_once(url, body, len) == ESP_OK) { g_post_ok++; return; }
        vTaskDelay(pdMS_TO_TICKS(delay_ms + (esp_random() % 250))); // jitter
        if (delay_ms < 8000) delay_ms *= 2;    // exponential, capped
    }
    g_post_fail++;
    ESP_LOGE("lab13b", "gave up after 5 attempts (ok=%" PRIu32 " fail=%" PRIu32 ")",
        g_post_ok, g_post_fail);
}

```

13B.5 Why each piece is there

.timeout_ms = 5000

Lab 13's config omitted this and took the default. An unbounded network call inside a task with any real-time responsibility is a latent hang. Bound every network operation.

the status check · if (status < 200 || status >= 300) err = ESP_FAIL;

This is the line that would have caught the original bug. `esp_http_client_perform()` returns `ESP_OK` for a completed HTTP exchange — including one that returned **501**. Lab 13 logged the status but did not act on it, so a totally failing setup still printed a cheerful success line.

exponential backoff with jitter

500 ms, then 1 s, 2 s, 4 s, 8 s. Backing off stops a dead server being hammered. The **jitter** — a random 0–250 ms added to each wait — matters when many devices are involved: without it, a fleet that all lost the server reconnects in perfect lockstep and knocks it over again the moment it returns. That failure mode has a name, the **thundering herd**, and it is a good thing to be able to name.

counters, not just logs

`g_post_ok` and `g_post_fail` make reliability *measurable*. A log line scrolls past; a counter you can read over BLE or print hourly tells you the actual failure rate. Every subsystem in a real product carries a few of these.

13B.6 Verify

1. Start the receiver. Confirm at least five POSTs return **HTTP 200** and print valid JSON on the laptop.
2. **Kill the receiver mid-run.** The board must log retries at growing intervals, then give up after five and increment `g_post_fail` — not hang, not reboot.
3. **Restart the receiver.** The next cycle must succeed with no manual intervention.
4. **Turn off the access point** for thirty seconds. The board must reconnect on its own and resume.

Step 2 is the one worth recording in `debug_log.md`. "I killed the server mid-run and the device backed off 0.5, 1, 2, 4, 8 seconds with jitter, then recovered without a reboot when it came back" is a concrete reliability story.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 13.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13 ←	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.